Solution: Maximum Distinct Elements

Problem Statement

Given an array of numbers and a number ‘K’, we need to remove ‘K’ numbers from the array such that we are left with maximum distinct numbers.

Example 1:

```
Input: [7, 3, 5, 8, 5, 3, 3], and K=2
Output: 3
Explanation: We can remove two occurrences of 3 to be left with 3
distinct numbers [7, 3, 8], we have
to skip 5 because it is not distinct and occurred twice.
Another solution could be to remove one instance of '5' and '3' each to
be left with three distinct numbers [7, 5, 8], in this case, we have to
skip 3 because it occurred twice.
```

Example 2:

```
Input: [3, 5, 12, 11, 12], and K=3
Output: 2
Explanation: We can remove one occurrence of 12, after which all numbers
will become distinct. Then we can delete any two numbers which will
leave us 2 distinct numbers in the result.
```

Example 3:

```
Input: [1, 2, 3, 3, 3, 3, 4, 4, 5, 5, 5], and K=2
Output: 3
Explanation: We can remove one occurrence of '4' to get three distinct
numbers.
```

Solution

This problem follows the `Top ‘K’ Numbers pattern`, and shares similarities with `Top ‘K’ Frequent Numbers`.

We can follow a similar approach as discussed in `Top ‘K’ Frequent Numbers` problem:

1. First, we will find the frequencies of all the numbers.
2. Then, push all numbers that are not distinct (i.e., have a frequency higher than one) in a **Min Heap** based on their frequencies. At the same time, we will keep a running count of all the distinct numbers.
3. Following a greedy approach, in a stepwise fashion, we will remove the least frequent number from the heap (i.e., the top element of the min-heap), and try to make it distinct. We will see if we can remove all occurrences of a number except one. If we can, we will increment our running count of distinct numbers. We have to also keep a count of how many removals we have done.
4. If after removing elements from the heap, we are still left with some deletions, we have to remove some distinct elements.

Code

Here is what our algorithm will look like:

Java

```java
import java.util.*;

class Solution {

  public int findMaximumDistinctElements(int[] nums, int k) {
    int distinctElementsCount = 0;
    if (nums.length <= k)
      return distinctElementsCount;

    // find the frequency of each number
    Map<Integer, Integer> numFrequencyMap = new HashMap<>();
    for (int i : nums)
      numFrequencyMap.put(i, numFrequencyMap.getOrDefault(i, 0) + 1);

    PriorityQueue<Map.Entry<Integer, Integer>> minHeap =
          new PriorityQueue<Map.Entry<Integer, Integer>>(
                (e1, e2) -> e1.getValue() - e2.getValue());

    // insert all numbers with frequency greater than '1' into the min-heap
    for (Map.Entry<Integer, Integer> entry : numFrequencyMap.entrySet()) {
      if (entry.getValue() == 1)
```

Run Code Result Stdout

You must run your code first

Execute

Time Complexity

Since we will insert all numbers in a **HashMap** and a **Min Heap**, this will take O(N∗logN) where ‘N’ is the total input numbers. While extracting numbers from the heap, in the worst case, we will need to take out ‘K’ numbers. This will happen when we have at least ‘K’ numbers with a frequency of two. Since the heap can have a maximum of ‘N/2’ numbers, therefore, extracting an element from the heap will take O(logN) and extracting ‘K’ numbers will take)O(KlogN). So overall, the time complexity of our algorithm will be $O(N*logN + KlogN)$.

We can optimize the above algorithm and only push ‘K’ elements in the heap, as in the worst case we will be extracting ‘K’ elements from the heap. This optimization will reduce the overall time complexity to $O(N*logK + KlogK)$.

Space Complexity

The space complexity will be O(N) as, in the worst case, we need to store all the ‘N’ characters in the **HashMap**.

← Previous
Maximum Distinct Elements (Medium)

Next →
Sum Of Elements (Medium)

☐ Mark as Completed